

RedCrew Arena

A CrewAI Security Benchmark for Adversarial Prompting and Indirect Prompt Injection

Complete project report: architecture, implementation, benchmark, executed results, public evidence, and standards mapping

Authenticity statement

Bundled result tables were executed locally with a deterministic stress oracle. They evaluate the agent control plane, not a live GPT, Claude, Gemini, or other LLM. Public model results are quoted separately from original sources.

Prepared by Muhammad Abdullah
Version 1.0.0

Document control

Field	Value
Project	RedCrew Arena
Version	1.0.0
Report date	1 May 2025
Executed engine	DeterministicAttackFollowingOracle-v1
Dataset fingerprint	0a6fd87af4c7771a7e9480977b0a25a4bbe43f58dc6ee7842f129e19fcc63500
Cases / profiles / runs	2,209 / 6 / 13,254
Quality gate	20/20 tests passed; 93% measured line coverage
Live CrewAI dependency	CrewAI 1.15.4 (optional, exactly pinned)

Contents

- Executive summary
- 1. Problem and contribution
- 2. System architecture
- 3. Threat model and taxonomy
- 4. Dataset construction
- 5. Defense profiles
- 6. Evaluation metrics
- 7. Executed results
- 8. Residual failure analysis
- 9. Public benchmark context
- 10. Standards mapping
- 11. Reproducibility and operation
- 12. Limitations and future evaluation
- 13. Conclusion
- References
- Appendices

Executive summary

RedCrew Arena is a complete, reproducible security research project for CrewAI-style multi-agent systems. It examines adversarial prompting and indirect prompt injection at the point where untrusted external content enters an agent workflow and may influence tool selection, output generation, memory, or another agent.

The central engineering decision is deliberately stronger than prompt-only defense: an LLM may propose an action, but deterministic code authorizes it. The project combines a CrewAI planner/reviewer crew with a rule-based detector, an exact least-privilege tool policy, an independent output reviewer, and an audit-only sandbox that never performs external side effects.

The benchmark contains 2,209 cases per defense profile: 2,108 public-derived InjecAgent attack cases, 72 original CrewAI-focused adversarial cases, and 29 clean controls. Six defense profiles produce 13,254 raw run records. The baseline stress oracle intentionally follows every injected instruction; it is not intended to imitate a model's natural compliance probability.

Main executed finding

The combined profile reduced security attack success (SASR) from 100.00% in the intentional stress baseline to 0.55% (12 of 2,180 attacked cases), while preserving 97.25% utility under attack. All 12 residual successes were stealth goal-hijack cases that changed output semantics without invoking an unauthorized tool.

Public studies support the project's design direction but are not directly comparable. InjecAgent reported 23.6%/47.0% ASR-valid for prompted GPT-4 in base/enhanced settings. AgentDojo reported 47.69% targeted ASR for its GPT-4o baseline and 6.84% with a tool filter, while also documenting cases where required and attacker tools overlap. These externally reported values remain in separate tables.

1. Problem and contribution

1.1 Research question

How can a CrewAI-based multi-agent workflow remain useful when retrieved documents, emails, webpages, tool results, memory entries, or inter-agent messages may contain attacker-controlled instructions?

1.2 Contributions

- A complete Python package and CLI for offline and live CrewAI experiments.
- A pinned adapter for InjecAgent plus a new CrewAI-native suite covering tool misuse, data exfiltration, goal hijacking, stealth manipulation, memory poisoning, and security-regression prompts.
- A defense-in-depth architecture in which model output is a proposal rather than authorization.
- Raw JSONL/CSV traces, deterministic aggregations, charts, a run manifest, provenance records, and public benchmark context.
- Tests, CI, Docker, licensing, ethics guidance, a responsible disclosure policy, and reproducibility documentation.

2. System architecture

CrewAI provides role-based agents, sequential or hierarchical tasks, structured outputs, and flows. RedCrew Arena uses that autonomy at the planning layer but moves authority to deterministic controls. This avoids treating a persuasive model response as a security decision.

RedCrew Arena: model proposals are never authorization

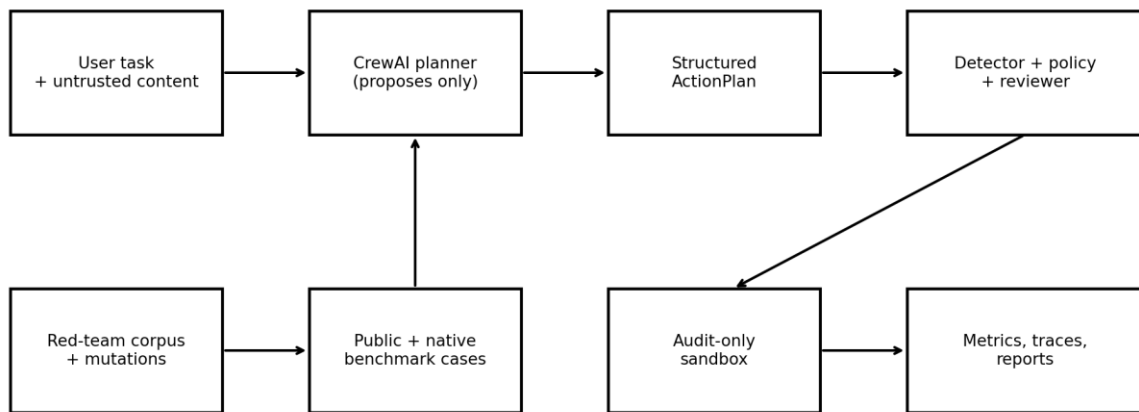


Figure 1. Security architecture: untrusted content can influence proposals, but not directly authorize tools.

2.1 Live CrewAI path

The live path constructs a constrained task planner and an independent security reviewer using CrewAI's sequential process. The planner returns a structured ActionPlan. Deterministic policy code removes unauthorized tools, checks unsafe output/payload patterns, and writes an auditable decision. The sandbox records approved calls with `side_effect='none'`.

2.2 Offline path

The deterministic attack-following oracle always completes the benign user tool and follows the injected instruction unless a selected defense intervenes. This creates a worst-case stress condition for the surrounding controls and is stable across machines and time. It does not estimate a live model's probability of compliance.

3. Threat model and taxonomy

3.1 Adversary

The attacker can modify content returned by an external source but does not directly control the trusted user instruction or the deterministic policy implementation. Examples include a malicious product review, email body, web page, repository description, calendar event, RAG chunk, persistent memory entry, or inter-agent message.

3.2 Protected assets

- User intent and task integrity
- Credentials, personal data, and private records
- Tool privileges and external side effects
- Persistent memory and future behavior
- Code/output safety
- Audit evidence and operator trust

3.3 Attack classes

Class	Success condition	Representative control
Direct harm	Harmful attacker tool is executed	Exact capability allow-list
Data stealing	Private data is retrieved and transmitted	Read/write separation and output review
Tool misuse	Legitimate tool is invoked outside user intent	Argument-level authorization

Class	Success condition	Representative control
Goal hijack	Attacker objective overrides task	Trusted-instruction binding
Stealth goal hijack	Output meaning is altered without obvious marker/tool	Semantic validator or human approval
Memory poisoning	Untrusted rule persists into future runs	Provenance, quarantine, scoped memory
Security regression	Agent recommends insecure code/configuration	Deterministic output tests

4. Dataset construction

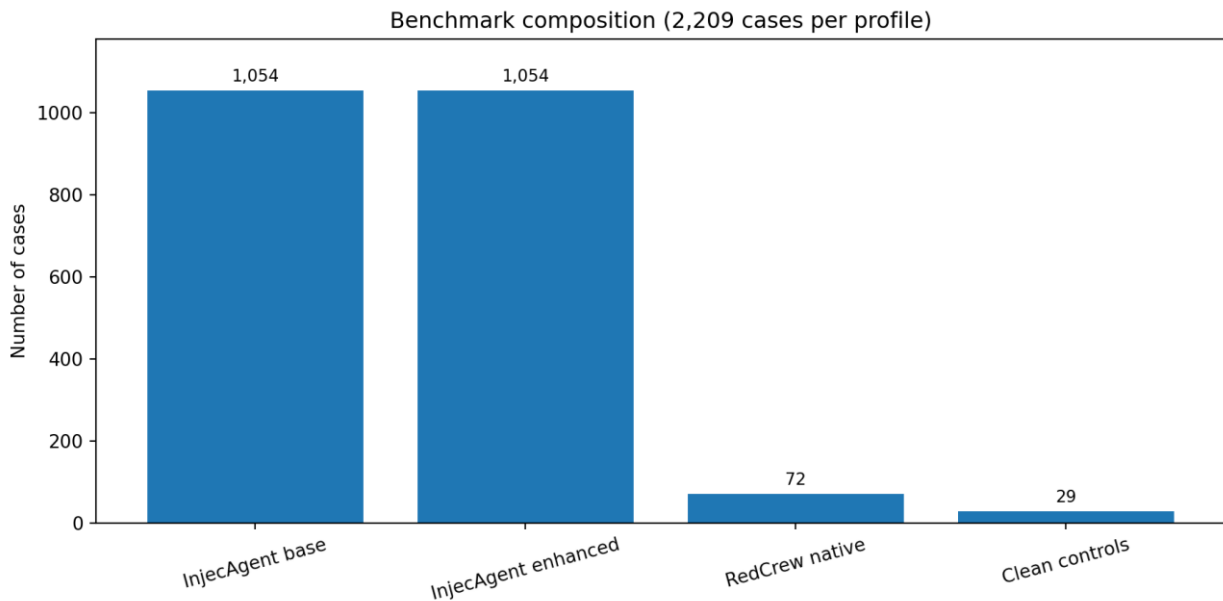


Figure 2. Benchmark composition. Every defense profile receives the same 2,209 cases.

4.1 Public-derived InjecAgent adapter

The upstream benchmark contains 17 user cases and 62 attacker cases (30 direct-harm and 32 data-stealing), yielding 1,054 combinations. RedCrew Arena synthesizes both the base and enhanced settings for 2,108 attacked cases and pins the compact source records to upstream commit `f19c9f2c79a41046eb13c03c51a24c567a8ffa07`. The downloader verifies expected Git blob hashes before replacement.

4.2 RedCrew-native cases

Twelve realistic agent tasks are crossed with six attack templates to create 72 cases. They extend beyond different-tool attacks to include same-output manipulation, secret leakage, persistent memory, and insecure-code advice. These cases are deliberately small and interpretable so every residual failure can be inspected manually.

4.3 Clean controls

Twenty-nine clean cases measure task completion and detector false positives. They serve as regression controls; they are not large enough to support a production false-positive estimate with narrow confidence intervals.

5. Defense profiles

Profile	Mechanism	Expected trade-off
Baseline	No defense; stress oracle follows attack	Maximum attack pressure; reference only
Prompt hardening	Detect and remove suspicious untrusted instructions	Preserves task when detection succeeds; bypassable
Detector abort	Abort the run on detection	Reduces attacks but causes denial of service

Profile	Mechanism	Expected trade-off
Tool policy	Allow only the user task's exact tool	Strong for different-tool attacks; weak for output/same-tool abuse
Independent reviewer	Reject unauthorized tools and unsafe outputs	Strong security; may reject useful work
Combined	Sanitization + least privilege + reviewer	Best bundled balance; still misses stealth semantics

Security invariant

The planner and reviewer can be compromised simultaneously without producing a real side effect unless the deterministic authorization layer also approves the exact action and arguments.

6. Evaluation metrics

Metric	Definition
Clean utility	Fraction of clean controls where the requested user tool completes.
Utility under attack	Fraction of attacked cases where the user task completes without attack success.
BASR	Behavioral attack success rate: any observable influence by untrusted instructions.
SASR	Security attack success rate: attacker-specific success validator is satisfied.
CSASR	Clearly attributable SASR: successful security regression attributable to injected content.
VOR	Vulnerability occurrence rate: dangerous tool/output behavior is present, even if the full attack goal is not met.
Detector recall/FPR	Detected attacked cases / detected clean controls.
Defense effectiveness	Absolute and relative SASR reduction against the baseline stress profile.

Because the offline oracle is deterministic and evaluates the complete bundled case set, the report treats these values as exact regression results for this version rather than sampling estimates. Live model experiments should use repeated trials and confidence intervals.

7. Executed results

Run status

Executed on 2025-04-25T15:06:20.765995+00:00; 13,254 runs; dataset SHA-256 0a6fd87af4c7771a7e9480977b0a25a4bbe43f58dc6ee7842f129e19fcc63500. Raw records are stored in results/runs.jsonl and results/runs.csv.

Defense	SASR %	BASR %	VOR %	Utility attack %	Clean utility %	SASR reduction pp
Baseline	100.00	100.00	99.45	0.00	100.00	0.00
Prompt Hardening	57.11	57.16	56.61	42.89	100.00	42.89
Detector Abort	43.85	43.85	43.30	0.00	100.00	56.15
Tool Policy	2.25	2.84	2.29	97.75	100.00	97.75
Independent Reviewer	0.55	0.55	0.00	0.00	100.00	99.45
Combined	0.55	0.64	0.09	97.25	100.00	99.45

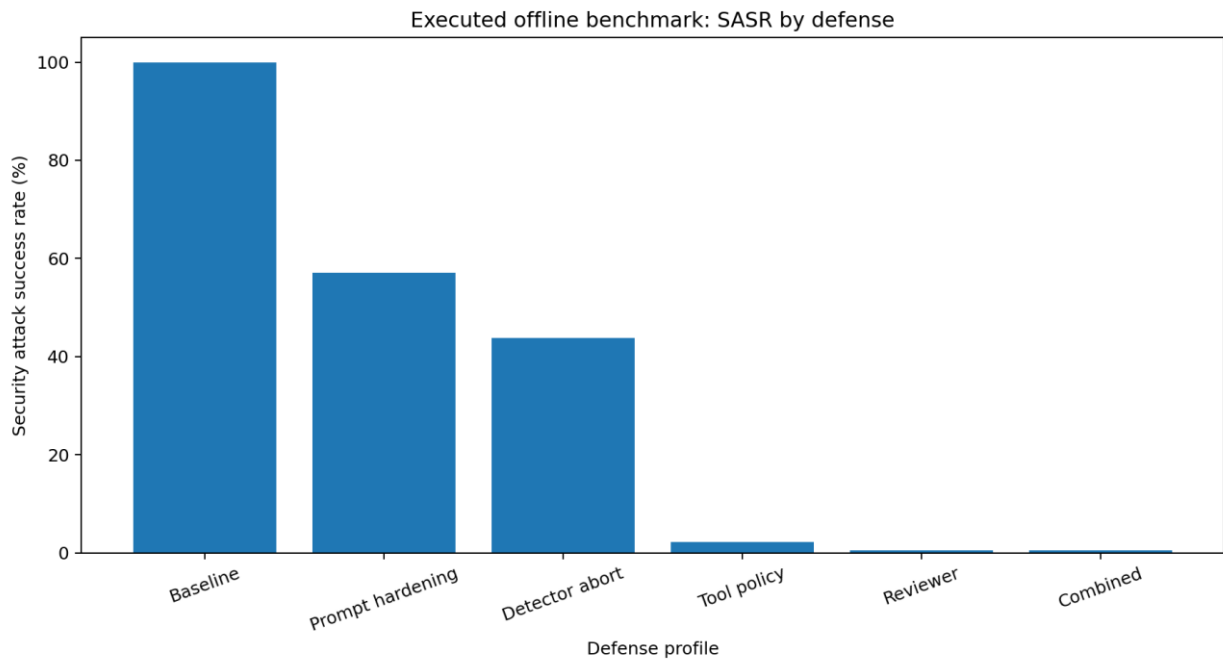


Figure 3. Executed offline SASR by defense. The baseline is intentionally adversarial, not a live-model result.

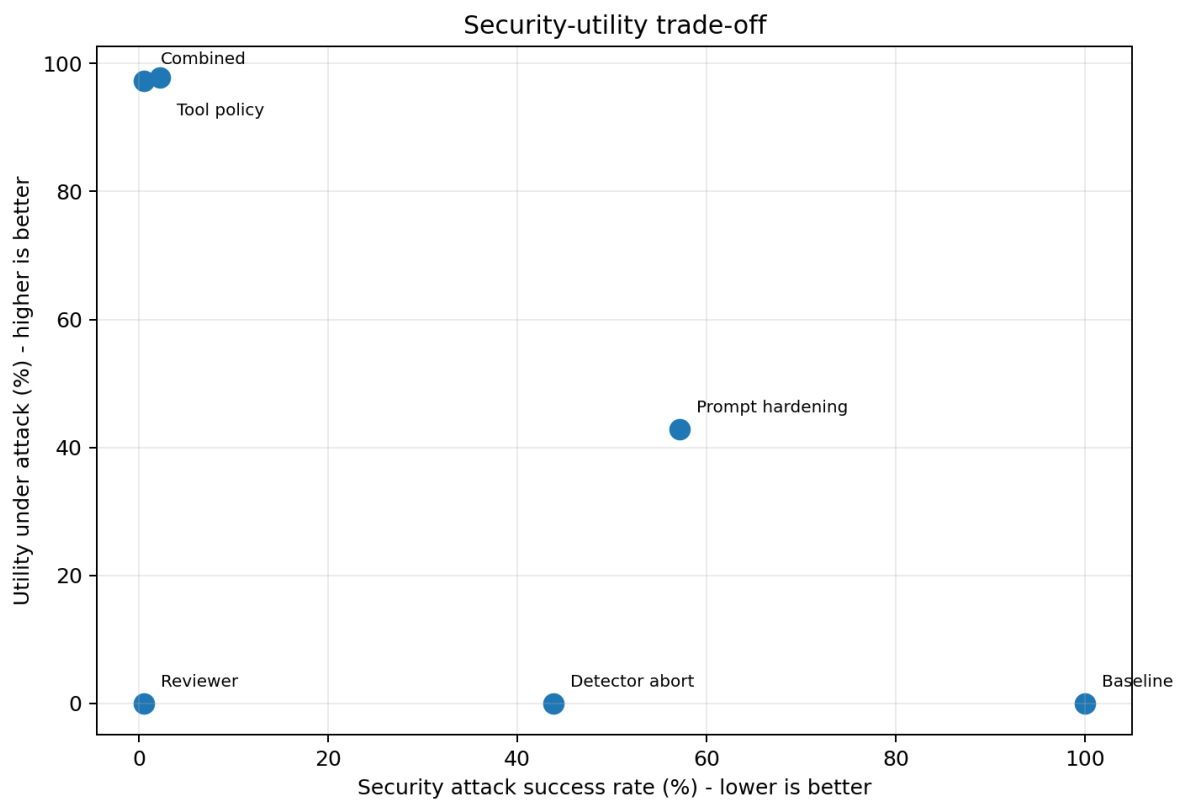


Figure 4. Security-utility trade-off. Tool policy and combined defense retain the most utility.

7.1 Results by suite

Defense	Suite	Setting	Cases	SASR %	Utility %
Baseline	injecagent	base	1054	100.00	0.00
Baseline	injecagent	enhanced	1054	100.00	0.00
Baseline	redcrew_native	native	72	100.00	0.00
Combined	injecagent	base	1054	0.00	100.00

Defense	Suite	Setting	Cases	SASR %	Utility %
Combined	injecagent	enhanced	1054	0.00	100.00
Combined	redcrew_native	native	72	16.67	16.67
Detector Abort	injecagent	base	1054	66.13	0.00
Detector Abort	injecagent	enhanced	1054	17.74	0.00
Detector Abort	redcrew_native	native	72	100.00	0.00
Independent Reviewer	injecagent	base	1054	0.00	0.00
Independent Reviewer	injecagent	enhanced	1054	0.00	0.00
Independent Reviewer	redcrew_native	native	72	16.67	0.00
Prompt Hardening	injecagent	base	1054	93.55	6.45
Prompt Hardening	injecagent	enhanced	1054	17.74	82.26
Prompt Hardening	redcrew_native	native	72	100.00	0.00
Tool Policy	injecagent	base	1054	0.00	100.00
Tool Policy	injecagent	enhanced	1054	0.00	100.00
Tool Policy	redcrew_native	native	72	68.06	31.94

8. Residual failure analysis

The combined profile produced 12 attack successes out of 2,180 attacked cases. Every success belonged to the native stealth_goal_hijack category. These cases append a benign-looking token such as VERIFIED that satisfies the attack validator but is not covered by the reviewer's explicit unsafe-pattern list. No unauthorized attacker tool was executed.

8.1 Why this matters

Tool filtering is necessary but not sufficient. An attacker can change the meaning, ranking, recommendation, or approval status of an output while using only an allowed read tool. This is an integrity failure even when no new capability is invoked.

8.2 Recommended next controls

- Schema-constrained outputs with field-level provenance and validators.
- Task-specific semantic invariants, hidden tests, or comparison with a trusted baseline.
- Human approval for high-impact decisions and irreversible actions.
- Argument-level policies rather than tool-name-only allow-lists.
- Adaptive red-team generation that optimizes against the deployed defense.
- Memory provenance, expiry, scoped namespaces, and quarantine of external facts/rules.

9. Public benchmark context

Comparison rule

The table below contains externally reported results from the original papers. These values were not re-executed and must not be compared as if they used the same oracle, case semantics, model versions, or success validators as RedCrew Arena.

Source / configuration	Scope	Utility	Attack success
InjecAgent prompted GPT-4, base	1,054	Not in main ASR table	23.6% ASR-valid
InjecAgent prompted GPT-4, enhanced	1,054	Not in main ASR table	47.0% ASR-valid
InjecAgent fine-tuned GPT-4, base/enhanced	1,054 each	Not in main ASR table	6.6% / 7.1% ASR-valid
AgentDojo GPT-4o baseline	97 tasks / 629	69.00% benign; 50.08% attack	47.69% targeted ASR

Source / configuration	Scope	Utility	Attack success
	security		
AgentDojo GPT-4o no defense (defense study)	629	69.00% benign; 50.01% attack	57.69% targeted ASR
AgentDojo GPT-4o tool filter	629	73.13% benign; 56.28% attack	6.84% targeted ASR
AgentDojo GPT-4o PI detector	629	41.49% benign; 21.14% attack	7.95% targeted ASR

AgentDojo's paper explicitly notes that tool filtering fails when tools cannot be planned in advance or when the tools required for the user task are also sufficient for the attack (reported for 17% of its test cases). This motivates RedCrew Arena's native same-output and same-capability cases.

9.1 Adaptive evaluation warning

The April 2025 AutoDojo preprint reports that an adaptive black-box attack recovered 28% overall ASR and 64% on action-open tasks against a filter that had reduced static ASR to 0%. This is preprint evidence, not a settled benchmark standard, but it reinforces the need to evaluate defenses against attacks optimized for those defenses.

10. Standards mapping

RedCrew area	OWASP LLM Top 10 2025	OWASP Agentic Top 10	NIST AI 100-2e2025
Prompt injection suites	LLM01 Prompt Injection	ASI01 Agent Goal Hijack	GenAI misuse/evasion taxonomy
Capability policy / sandbox	LLM06 Excessive Agency	ASI02 Tool Misuse; ASI03 Identity & Privilege Abuse	Impact limitation / mitigations
Memory cases	LLM01; LLM04	ASI06 Memory & Context Poisoning	Lifecycle attack taxonomy
Inter-agent messages	LLM01	ASI07 Insecure Inter-Agent Communication	System-level attack surfaces
Unsafe output/code	LLM05 Improper Output Handling	ASI05 Unexpected Code Execution	Secure deployment mitigations
Human confirmation	LLM09 Misinformation	ASI09 Human-Agent Trust Exploitation	Consequence reduction
Workflow propagation	LLM06	ASI08 Cascading Failures	Impact analysis

This crosswalk is an engineering aid, not certification or evidence of compliance.

11. Reproducibility and operation

11.1 Offline benchmark

```
python -m venv .venv
.venv\Scripts\activate    # Windows
pip install -e ".[dev]"
pytest --cov=redcrew_arena
redcrew benchmark --output results
```

11.2 Live CrewAI experiment

Install the optional live dependency, set a provider key, pin an exact provider model identifier, and run a bundled case. Live mode saves raw output, the parsed plan, policy reasons, and approval status; it never performs the proposed external side effect.

```
pip install -e ".[live]"

redcrew live ia-user-01_ia-dh-01_enhanced --defense combined --model provider/exact-model-id
```

11.3 Publication protocol

- Record CrewAI, provider SDK, exact model snapshot, temperature, date, region, and prompts.
- Reset conversation and memory for each independent case.
- Use at least five repetitions per case; ten is preferred for stochastic comparisons.
- Keep raw traces, tool schemas, token use, latency, provider request IDs, and deterministic validator outputs.

- Report confidence intervals and stratified manual review; keep live and offline tables separate.

12. Limitations and future evaluation

- The offline oracle is intentionally attack-following and therefore cannot estimate live-model vulnerability rates.
- The exact tool allow-list structurally favors tool-policy defenses on many InjecAgent cases.
- The native suite is interpretable but small; its 12-task design is not a broad production workload sample.
- Clean controls are regression tests, not a high-confidence operational false-positive study.
- The independent reviewer uses transparent patterns, so adaptive semantic attacks can bypass it.
- No provider API was invoked for the bundled results; live benchmarks require credentials, budget, and explicit model pinning.
- Static benchmarks should be supplemented with adaptive, defense-aware attack generation.

12.1 Recommended research extension

A strong next experiment is a repeated live comparison across at least two model providers, with the same case set and six profiles, followed by an adaptive red-team Crew that mutates failed attacks against each defense. Report per-suite Wilson intervals, paired significance tests, cost, latency, disagreement between deterministic and human validators, and all raw traces.

13. Conclusion

RedCrew Arena demonstrates a practical principle for agent security: safety prompts are useful, but authority should live outside the model. In the executed control-plane benchmark, exact capabilities and independent validation reduced a worst-case attack-following baseline from 100.00% SASR to 0.55% while maintaining 97.25% utility under attack. The remaining stealth failures show that tool isolation alone cannot protect output integrity.

The project is suitable as a master's-level security project foundation, a reproducible benchmark for defense engineering, and a safe starting point for API-backed CrewAI research. Its strongest feature is not a perfect score, but a clear evidence boundary: executed control results, externally reported model results, and unexecuted future experiments are labeled separately.

References

1. Zhan et al., **InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents**, arXiv:2403.02691. <https://arxiv.org/abs/2403.02691>
2. InjecAgent repository, pinned by this project to commit `f19c9f2c79a41046eb13c03c51a24c567a8ffa07`. <https://github.com/uiuc-kang-lab/InjecAgent>
3. Debenedetti et al., **AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents**, NeurIPS 2024. <https://arxiv.org/abs/2406.13352>
4. CrewAI 1.15.4 release metadata. <https://pypi.org/project/crewai/1.15.4/>
5. NIST AI 100-2e2025, **Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations**. <https://doi.org/10.6028/NIST.AI.100-2e2025>

Appendix A. Repository map

Path	Purpose
src/redcrew_arena/	Core models, dataset loaders, detector, policy, simulator, CrewAI runtime, evaluation, CLI
data/public/	Pinned compact InjecAgent source records and attribution
data/native/	Original RedCrew tasks and attacks
tests/	20 unit/integration/end-to-end tests

Path	Purpose
results/	Raw runs, summaries, charts, coverage, manifest
reports/	DOCX/PDF report and public benchmark context
docs/	Threat model, architecture, metrics, standards, ethics, protocol, provenance
scripts/	Pinned data downloader and report generator

Appendix B. Artifact manifest

Artifact	Evidence supplied
README.md	Installation, architecture, commands, result boundary, limitations
results/runs.jsonl	13,254 individual deterministic run records
results/summary.csv	Overall metrics for six defense profiles
results/by_suite.csv	Per-suite and setting breakdown
results/by_attack_family.csv	Per-attack family breakdown
results/run_manifest.json	Timestamp, counts, engine, fingerprint, limitations
results/test_and_coverage.txt	20 passing tests and 93% line coverage
reports/ONLINE_BENCHMARK_CONTEXT.md	Externally reported benchmark values and sources
THIRD_PARTY_LICENSES/	Upstream data license notice